UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Network Security

Elaborato Network Security

Laboratorio di simulazione di attività ransomware

Studente
Francescomaria Pierro
Matricola: [M63001870]

Abstract

Il presente elaborato descrive la progettazione, l'implementazione e la verifica sperimentale di un laboratorio di Network Security finalizzato al rilevamento di attività sospette e all'applicazione automatica di contromisure di rete. L'ambiente è stato realizzato mediante container Docker distribuiti su due sottoreti, collegate attraverso un Gateway sul quale operano Suricata, nftables e un motore di analisi comportamentale sviluppato in Python.

Il sistema integra il rilevamento basato su firme con l'analisi delle richieste applicative su finestre temporali di diversa durata. Quando una sorgente soddisfa le condizioni di rilevamento configurate, il Gateway può reindirizzarne temporaneamente le connessioni verso un honeypot, mantenendo separato il servizio legittimo. L'architettura comprende inoltre un componente per il monitoraggio del filesystem e uno strumento di correlazione offline degli eventi.

La verifica sperimentale ha riguardato i singoli componenti, il confronto tra due versioni del motore comportamentale e una prova integrata del meccanismo di risposta automatica. Nei quattro scenari sintetici considerati, la seconda versione ha rilevato entrambi i comportamenti classificati come sospetti, mentre la prima ne ha individuato uno soltanto. La prova end-to-end ha inoltre mostrato l'attivazione del reindirizzamento verso l'honeypot e il successivo ripristino del servizio legittimo alla scadenza del timeout.

I risultati documentano il funzionamento dei meccanismi implementati nelle condizioni sperimentali considerate, ma non consentono di valutarne l'efficacia contro attacchi ransomware reali. La correlazione temporale tra eventi di rete e modifiche del filesystem non permette, da sola, di stabilire un rapporto causale.

Indice

Abstract **1**

1 Introduzione **3**
1.1 Contesto e motivazioni del progetto 3
1.2 Obiettivi 3
1.3 Approccio progettuale 4

2 Architettura del laboratorio **5**
2.1 Struttura e topologia 5
2.2 Ruolo del Gateway 6
2.3 Servizio legittimo e honeypot 6
2.4 FileMonitor, log ed evidenze 6

3 Implementazione **8**
3.1 Organizzazione del progetto 8
3.2 Configurazione del Gateway e risposta tramite nftables 8
3.3 Rilevamento basato su firme con Suricata 8
3.4 Behavioral Engine 9
3.5 Integrazione e avvio automatico 9
3.6 Honeypot, FileMonitor e correlatore 10

4 Metodologia sperimentale **11**
4.1 Obiettivi della verifica sperimentale 11
4.2 Tipologie di test 11
4.3 Criteri di valutazione 12
4.4 Prova end-to-end 12

5 Risultati sperimentali **13**
5.1 Behavioral Engine V1 e V2 13
5.2 FileMonitor e correlatore 14
5.3 Avvio automatico del Gateway 14
5.4 Prova end-to-end 14

6 Discussione e limiti **16**
6.1 Interpretazione dei risultati 16
6.2 Vantaggi dell'architettura 16
6.3 Limiti 16
6.4 Possibili sviluppi futuri 17
6.5 Sintesi 17

7 Riproduzione degli esperimenti **18**
7.1 Obiettivo e requisiti 18
7.2 Preparazione e avvio 18
7.3 Verifiche iniziali 19
7.4 Traffico di test e rilevamento 19
7.5 Reindirizzamento e timeout 20
7.6 Evidenze 20

Capitolo 1

Introduzione

1.1 Contesto e motivazioni del progetto

Il rilevamento delle attività sospette costituisce un aspetto fondamentale della sicurezza delle reti informatiche. Tuttavia, la sola identificazione di un comportamento potenzialmente malevolo non è sempre sufficiente a limitarne le conseguenze. In determinati scenari risulta quindi utile affiancare ai meccanismi di monitoraggio strumenti in grado di intervenire automaticamente sul traffico.

I sistemi di Intrusion Detection (IDS) analizzano il traffico o gli eventi di sistema per individuare possibili violazioni delle politiche di sicurezza. I sistemi di Intrusion Prevention (IPS) estendono tali funzionalità mediante contromisure, tra cui il blocco e il reindirizzamento delle connessioni. Il rilevamento basato su firme presenta tuttavia dei limiti quando un comportamento sospetto emerge dalla successione delle richieste anziché dal contenuto di una singola comunicazione. Per esaminare questo caso, nel laboratorio è stato affiancato a Suricata un motore che considera la frequenza e la distribuzione temporale delle richieste applicative.

Lo scenario sperimentale richiama alcuni comportamenti associabili ad attività ransomware, senza prevedere l'esecuzione di codice malevolo o la cifratura effettiva dei documenti. L'obiettivo è osservare l'integrazione tra rilevamento e risposta automatica in un ambiente controllato.

1.2 Obiettivi

L'obiettivo è avere un ambiente sperimentale in cui osservare come interagiscono rilevamento, analisi degli eventi e risposta automatica. In concreto, il lavoro comprende:

- una rete di laboratorio con container Docker separati, il cui traffico passa da un Gateway dedicato;
- Suricata per il rilevamento a firme e un motore comportamentale per l'analisi delle richieste applicative;
- una risposta temporanea basata su regole nftables, con un honeypot che registra le interazioni ricevute;
- un monitor del filesystem, i cui eventi vengono confrontati nel tempo con quelli di rete;
- una verifica sperimentale del funzionamento e una discussione dei limiti.

1.3 Approccio progettuale

Il laboratorio è stato sviluppato su Kali Linux, utilizzando Docker Compose per definire container e reti. I componenti sono quattro: Attacker, Gateaway, Target e FileMonitor. Il Gateway costituisce il punto centrale dell'architettura: instrada il traffico tra le reti e ospita Suricata, nftables e il motore comportamentale. Sul Target convivono il servizio legittimo e l'honeypot; in condizioni ordinarie le richieste raggiungono il primo, mentre, in presenza delle condizioni di rilevamento previste, il traffico della sorgente interessata viene reindirizzato temporaneamente al secondo.

Il motore comportamentale è stato sviluppato in due versioni successive. La seconda aggiunge una finestra temporale più lunga, per rilevare anche attività meno concentrate nel tempo. La verifica comprende test dei singoli componenti, prove di correlazione su eventi controllati e una verifica integrata della risposta automatica.

Capitolo 2

Architettura del laboratorio

2.1 Struttura e topologia

Il laboratorio simula la comunicazione tra un nodo che genera richieste e un servizio di destinazione, attraverso un Gateway intermedio, responsabile del monitoraggio e dell'applicazione delle regole di rete. I quattro container hanno ruoli distinti:

- **Attacker** genera le richieste previste dagli scenari;
- **Gateaway** collega le due sottoreti e ospita rilevamento e risposta;
- **Target** offre il servizio legittimo e l'honeypot;
- **FileMonitor** osserva gli eventi del filesystem.

La denominazione `Gateaway` è mantenuta per coerenza con la configurazione del progetto; nel testo il componente è indicato anche come "Gateway". La separazione dei ruoli consente di verificare prima i singoli componenti e successivamente il funzionamento integrato del sistema.

Le reti Docker sono due: `internal_net` (`10.10.10.0/24`), alla quale appartiene l'Attacker, e `service_net` (`10.20.20.0/24`), alla quale appartiene il Target. Il Gateway ha un'interfaccia in ciascuna rete, e rappresenta il punto di transito del traffico tra le sottoreti, consentendo di centralizzarne il controllo.

Tabella 2.1: Indirizzamento dei componenti di rete

Componente	**Rete**	**Indirizzo IP**
Attacker	internal_net	10.10.10.2
Gateaway	internal_net	10.10.10.3
Gateaway	service_net	10.20.20.3
Target	service_net	10.20.20.2

La Figura 2.1 mostra la struttura logica. Il FileMonitor è rappresentato separatamente per evidenziarne il ruolo di osservazione degli eventi del filesystem.

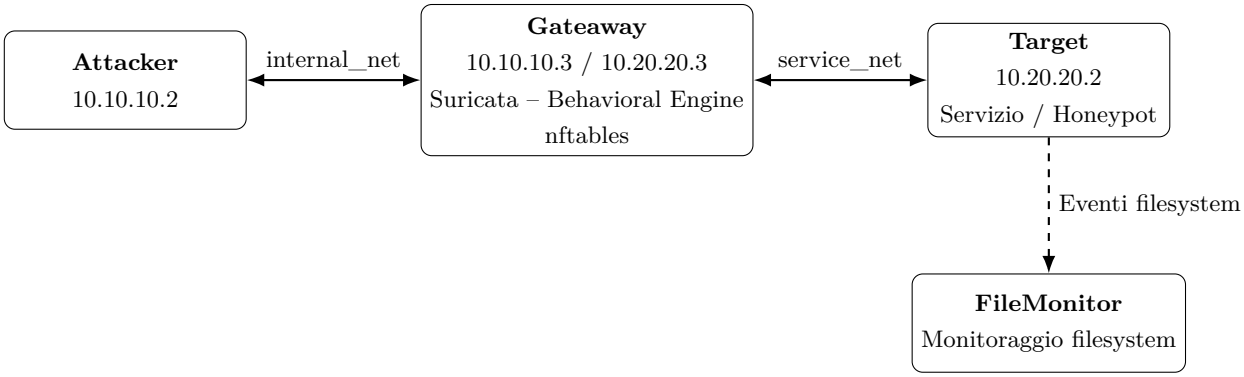


Figura 2.1: Architettura logica del laboratorio

2.2 Ruolo del Gateway

Il Gateway non si limita a instradare i pacchetti. Ospita tre elementi: **Suricata**, che analizza il traffico e genera segnalazioni in base alle regole; il **Behavioral Engine**, scritto in Python, che individua sequenze di richieste considerate sospette; e **nftables**, che gestisce filtraggio, blocco temporaneo e reindirizzamento.

Il motore può aggiornare in modo dinamico gli insiemi di indirizzi IP di nftables, quindi il rilevamento di una sequenza sospetta si traduce in un cambiamento temporaneo nel trattamento del traffico di quella sorgente. Gli elementi inseriti negli insiemi hanno un timeout, per cui il comportamento normale si ripristina senza richiedere un intervento manuale.

2.3 Servizio legittimo e honeypot

Il Target espone due servizi. Il servizio legittimo risponde sulla porta TCP 8081, l'honeypot sulla 8080. In condizioni normali una richiesta alla 8081 raggiunge il servizio legittimo; se il motore identifica una sorgente come sospetta, il Gateway applica una regola di Destination Network Address Translation (DNAT) che reindirizza le connessioni successive verso la porta 8080.

I due servizi sono distinguibili attraverso le rispettive risposte: il servizio legittimo restituisce `LAB_LEGITIMATE_SERVICE`, l'honeypot un'intestazione `DOCUMENT-SERVICE/1.0`. Durante i test, l'analisi della risposta permette quindi di verificare quale servizio abbia effettivamente gestito la richiesta. L'honeypot non costituisce un ambiente di esecuzione di ransomware: è un servizio controllato che registra le richieste reindirizzate.

2.4 FileMonitor, log ed evidenze

Il FileMonitor registra gli eventi sul filesystem, in particolare creazioni e rinomine di file, e serve a costruire scenari sintetici che richiamano una modifica massiva di documenti. È una fonte diversa dal traffico di rete: un evento del filesystem non identifica automaticamente la sorgente IP responsabile dell'operazione. Per questo l'eventuale legame tra eventi di rete e di filesystem viene considerato una correlazione temporale e non una dimostrazione di causalità.

I componenti producono log distinti: Suricata gli eventi sul traffico, il motore le decisioni e le azioni di risposta, l'honeypot le interazioni, il FileMonitor gli eventi sui file. I documenti del Target sono conservati

in un volume Docker denominato `ransomware_demo_data`, separato dallo stato del container, per favorire la ripetibilità degli esperimenti e la conservazione delle evidenze.

Capitolo 3

Implementazione

3.1 Organizzazione del progetto

La configurazione dell'infrastruttura (container, reti, volumi, collegamenti) è definita nel file `docker-compose.yml`; gli script di avvio e i programmi Python sono organizzati nelle directory dei rispettivi componenti. Tale struttura separa la logica applicativa dalla configurazione dell'infrastruttura e dalle procedure sperimentali. L'avvio del laboratorio avviene mediante `scripts/start-lab.sh`, che avvia i container e predispone le regole di rete.

3.2 Configurazione del Gateway e risposta tramite nftables

Il Gateway è configurato come nodo intermedio, con le rotte necessarie tra Attacker e Target. Il filtraggio è affidato a nftables: la tabella `inet lab_filter` ha una catena di forwarding con politica predefinita di blocco ed eccezioni per le connessioni già stabilite e per il traffico autorizzato dal laboratorio. In questo modo l'inoltro non è indiscriminatamente permissivo. Le regole vengono create dallo script di avvio, mentre i processi di rilevamento vengono avviati nel container Gateway.

La risposta automatica usa due insiemi dinamici di indirizzi IP, entrambi con timeout di un minuto:

- `blocked_v4`, nella tabella `inet ips_response`, per il blocco temporaneo;
- `suspicious_v4`, nella tabella `ip lab_redirect`, per selezionare le sorgenti da reindirizzare.

Quando una sorgente è in `suspicious_v4`, il Gateway applica il DNAT che porta le sue connessioni alla porta 8081 del Target sulla 8080, cioè sull'honeypot. L'implementazione prevede due modalità di risposta: il blocco temporaneo delle connessioni e il reindirizzamento verso l'honeypot. Quest'ultima modalità consente di osservare le interazioni successive della sorgente sospetta all'interno di un ambiente controllato.

3.3 Rilevamento basato su firme con Suricata

Suricata è un motore open source di analisi del traffico. Nel laboratorio è stata utilizzata la versione 7.0.10, configurata sull'interfaccia del Gateway verso la sottorete dei servizi, con regole scritte per gli scenari di

prova. La regola `SID 1000003` segnala le richieste HTTP GET; la `SID 1000004` riconosce il contenuto applicativo `SIMULATE_ENCRYPT` e ne prevede il blocco. Il contenuto costituisce un marcatore sintetico impiegato nei test e non comporta alcuna cifratura reale dei documenti.

Le firme permettono di riconoscere condizioni specifiche, ma una sequenza di richieste può risultare anomala anche in assenza di indicatori esplicitamente malevoli nelle singole richieste. Questa limitazione motiva l'impiego del motore comportamentale.

3.4 Behavioral Engine

Il Behavioral Engine è un programma Python che, a differenza delle firme, valuta il comportamento di una sorgente dentro finestre temporali. Quando le condizioni previste per gli scenari del laboratorio sono soddisfatte, produce un evento con la sorgente, la condizione riconosciuta e il punteggio assegnato.

Versione 1. La prima versione è orientata al rilevamento di accessi a più documenti concentrati in un intervallo temporale breve. Tale impostazione risulta adatta a sequenze particolarmente intense, ma può non rilevare attività distribuite su intervalli più ampi. Questo limite ha motivato lo sviluppo della seconda versione.

Versione 2. La seconda versione conserva la condizione di rilevamento rapido e introduce una finestra temporale estesa, prevedendo complessivamente due condizioni (Tabella 3.1). La condizione `fast_10s` usa una finestra di 10 secondi e assegna 100 punti; `extended_60s` usa una finestra di 60 secondi e ne assegna 60.

Tabella 3.1: Condizioni di rilevamento del Behavioral Engine V2

Condizione	**Finestra**	**Punteggio**
fast_10s	10 secondi	100
extended_60s	60 secondi	60

I punteggi sono valori di progetto, non probabilità di compromissione né misure generali della pericolosità del traffico.

3.5 Integrazione e avvio automatico

Lo script `gateway/start-gateway.sh` avvia Suricata e il processo Python del motore. Il motore viene eseguito con l'opzione `-apply`, che abilita le azioni previste, e con un intervallo di interrogazione di 0,2 secondi. Lo script attende che gli insiemi nftables siano disponibili prima dell'avvio del motore, riducendo il rischio di aggiornare strutture non ancora inizializzate. È inoltre previsto il controllo dei processi per rilevare eventuali interruzioni. Il funzionamento dell'avvio automatico è stato verificato ricreando il container Gateway e ripetendo la prova.

3.6 Honeypot, FileMonitor e correlatore

L'honeypot è un servizio applicativo sulla porta TCP 8080 che riceve le richieste reindirizzate e ne registra le informazioni utili. La sua risposta identificativa permette di controllare il DNAT senza strumenti di ispezione complessi.

Il FileMonitor registra creazioni e rinomine di documenti. Nello scenario sintetico di verifica sono state considerate 15 creazioni e 15 rinomine, che hanno prodotto un punteggio pari a 90 secondo la logica configurata.

Il correlatore è uno strumento offline che confronta i log mediante identificativi, indirizzi IP e riferimenti temporali, per individuare possibili relazioni tra eventi di componenti diversi entro intervalli prestabiliti. Lo strumento facilita la ricostruzione degli scenari, ma non dimostra un rapporto causale tra gli eventi. Per tale attribuzione sarebbero necessari ulteriori meccanismi di tracciamento dei processi e delle operazioni eseguite.

Le evidenze (log, configurazioni, esportazioni dello stato nftables) sono conservate prevalentemente nella directory `evidence/gateway-autostart/`, e permettono di confrontare il comportamento atteso con quello osservato.

Capitolo 4

Metodologia sperimentale

4.1 Obiettivi della verifica sperimentale

La verifica sperimentale non è finalizzata a misurare l'efficacia del sistema contro un insieme rappresentativo di attacchi ransomware reali, bensì a controllare il funzionamento dei meccanismi implementati in scenari riproducibili. L'analisi riguarda il rilevamento delle attività sospette, l'applicazione delle contromisure di rete e la registrazione degli eventi, con particolare attenzione alla modifica temporanea del percorso del traffico in seguito all'identificazione di una sorgente sospetta.

Gli esperimenti sono stati eseguiti su Kali Linux con Docker e Docker Compose. Le richieste di test partono dal container Attacker (`10.10.10.2`) verso il Target; il servizio legittimo è sulla 8081 e l'honeypot sulla 8080. Il contesto containerizzato mantiene costante la configurazione di rete e permette di ripetere le prove senza coinvolgere sistemi esterni.

4.2 Tipologie di test

Connettività. Controllo di rotte e regole nftables create all'avvio e verifica che una richiesta alla porta 8081 raggiunga il servizio legittimo, con risposta `LAB_LEGITIMATE_SERVICE`. Questa risposta fa da riferimento per distinguere il comportamento normale da quello dopo il reindirizzamento.

Suricata. Generazione di traffico applicativo controllato per verificare che le regole sulle richieste GET e sul marcatore `SIMULATE_ENCRYPT` producano gli eventi attesi, controllati poi nei log. Nessuna cifratura reale viene eseguita.

Motore comportamentale. Valutazione su scenari sintetici con esito noto, confrontando V1 e V2. Gli scenari comprendono attività sospette concentrate, attività sospette distribuite su un intervallo più ampio e casi non sospetti, per controllare che non ci siano segnalazioni quando le condizioni non si verificano.

FileMonitor. Scenario con 15 creazioni e 15 rinomine, per controllare la produzione degli eventi e il calcolo del punteggio.

Correlatore. Quattro test sintetici con informazioni temporali e identificative utilizzabili per il confronto.

4.3 Criteri di valutazione

Per il confronto tra V1 e V2, ciascuno scenario è stato classificato come True Positive (sospetto e rilevato), False Negative (sospetto non rilevato), True Negative (non sospetto e ignorato) o False Positive (non sospetto ma segnalato), e i risultati sono stati organizzati in una matrice di confusione per ciascuna versione. Il campione comprende quattro scenari, due sospetti e due non sospetti. La dimensione ridotta consente una verifica funzionale nei casi selezionati, ma non una stima rappresentativa delle prestazioni su traffico reale.

4.4 Prova end-to-end

La prova integrata controlla l'intera sequenza ed è stata eseguita dopo la ricreazione del container Gateway, per includere anche la verifica dell'avvio automatico. La procedura comprende le seguenti fasi:

1. controllo dello stato iniziale delle regole nftables e dell'insieme `suspicious_v4`;
2. verifica che il servizio legittimo sulla 8081 sia raggiungibile;
3. generazione di 15 richieste HTTP GET dall'Attacker;
4. controllo degli eventi prodotti dal Behavioral Engine;
5. verifica che l'indirizzo sorgente sia stato inserito in `suspicious_v4`;
6. nuova richiesta alla 8081 e controllo della risposta;
7. attesa della scadenza del timeout;
8. verifica che l'indirizzo sia stato rimosso e che il servizio legittimo risponda di nuovo.

Il rilevamento atteso è quello della condizione `fast_10s`, che richiede accessi a documenti distinti entro la finestra prevista.

Capitolo 5

Risultati sperimentali

5.1 Behavioral Engine V1 e V2

I risultati del confronto sono nella Tabella 5.1.

Tabella 5.1: Confronto dei risultati del Behavioral Engine V1 e V2

Indicatore	**V1**	**V2**
True Positive (TP)	1	2
False Negative (FN)	1	0
True Negative (TN)	2	2
False Positive (FP)	0	0
Totale scenari	4	4

La V1 ha riconosciuto uno dei due scenari sospetti e ha mancato l'altro; la V2 li ha riconosciuti entrambi. Entrambe le versioni hanno classificato correttamente i due scenari non sospetti, senza falsi positivi.

Dalla matrice si ricavano tre metriche:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{5.1}$$

Tabella 5.2: Metriche descrittive sui quattro scenari sintetici

Metrica	**V1**	**V2**
Precision	100%	100%
Recall	50%	100%
Accuracy	75%	100%

Il miglioramento riguarda soprattutto il recall, dal 50% al 100%, ed è coerente con l'aggiunta di `extended_60s`, pensata per attività distribuite su tempi più lunghi. Le percentuali, però, vengono da quattro scenari soltanto.

5.2 FileMonitor e correlatore

Nello scenario sintetico con 15 creazioni e 15 rinomine il FileMonitor ha prodotto il punteggio previsto di 90 (Tabella 5.3).

Tabella 5.3: Risultato dello scenario sintetico del FileMonitor

Indicatore	Valore
Creazioni di file	15
Rinomine di file	15
Punteggio assegnato	90

Il correlatore ha superato tutti e quattro i test (Tabella 5.4).

Tabella 5.4: Risultati dei test del correlatore

Indicatore	Valore
Test eseguiti	4
Test superati	4
Test non superati	0

5.3 Avvio automatico del Gateway

Dopo la ricreazione del container Gateway ho controllato che partissero Suricata e il Behavioral Engine e che le strutture nftables fossero disponibili, senza lanciare il motore a mano. Il controllo è utile perché il fatto che i singoli programmi funzionino non garantisce che si integrino bene all'avvio. Configurazione e stato del Gateway sono tra le evidenze conservate.

5.4 Prova end-to-end

Stato iniziale. L'insieme `suspicious_v4` non conteneva l'indirizzo dell'Attacker e una richiesta alla porta 8081 del Target raggiungeva il servizio legittimo, con risposta:

```
LAB_LEGITIMATE_SERVICE
```

Richieste e rilevamento. Dall'Attacker (`10.10.10.2`) ho generato 15 richieste HTTP GET. Si è attivata la condizione `fast_10s`: l'evento registrato riporta 10 richieste GET verso documenti distinti in una finestra di 10 secondi, con punteggio 100 e UUID:

```
688872c0-a32c-49d2-ae99-46f3f514be63
```

Reindirizzamento. L'indirizzo `10.10.10.2` è stato inserito in `suspicious_v4`, con circa 58 secondi di tempo residuo, in linea con il timeout di un minuto, e questo ha attivato la regola DNAT. Una nuova richiesta alla 8081 non ha più raggiunto il servizio legittimo ma l'honeypot, che ha risposto:

```
DOCUMENT-SERVICE/1.0
```

Scadenza e ripristino. Dopo circa 65 secondi di attesa l'indirizzo non era più nell'insieme e una nuova richiesta alla 8081 ha restituito di nuovo `LAB_LEGITIMATE_SERVICE`: il reindirizzamento era stato rimosso e il percorso ordinario ripristinato.

Tabella 5.5: Sintesi della verifica end-to-end

Fase	**Risultato osservato**
Stato iniziale	Insieme suspicious_v4 vuoto; servizio legittimo raggiungibile
Generazione del traffico	15 tentativi HTTP GET dal container Attacker
Rilevamento	Condizione fast_10s, punteggio 100
Risposta automatica	Inserimento di 10.10.10.2 nell'insieme suspicious_v4
Reindirizzamento	Risposta dell'honeypot sulla connessione diretta alla porta 8081
Scadenza	Rimozione dell'indirizzo dopo il timeout
Ripristino	Nuova risposta del servizio legittimo

Capitolo 6

Discussione e limiti

6.1 Interpretazione dei risultati

Il risultato più significativo è la prova end-to-end: una sequenza di richieste HTTP ha attivato il Behavioral Engine, poi il reindirizzamento verso l'honeypot, e infine, dopo il timeout, le richieste sono tornate al servizio legittimo. La contromisura viene rimossa automaticamente alla scadenza del timeout configurato. In secondo luogo, usare due finestre temporali (una rapida e una più ampia) si è rivelato utile nei casi preparati, perché permette di cogliere sia sequenze veloci sia richieste più distanziate.

Suricata e il motore comportamentale operano su criteri differenti: il primo rileva indicatori definiti dalle firme, mentre il secondo analizza la distribuzione temporale delle richieste. I due approcci risultano pertanto complementari sul piano funzionale. Non è stata tuttavia condotta una valutazione statistica dell'efficacia combinata, per cui non è possibile quantificare il beneficio rispetto all'impiego di un solo meccanismo.

6.2 Vantaggi dell'architettura

La suddivisione del laboratorio in container facilita l'analisi dei singoli componenti. L'instradamento attraverso il Gateway centralizza monitoraggio e risposta, consentendo di verificare le decisioni di filtraggio e reindirizzamento senza modificare il container Attacker. Il timeout degli insiemi nftables evita che le contromisure restino attive indefinitamente, pur non eliminando il rischio di penalizzare traffico legittimo classificato erroneamente. Infine Docker Compose, gli script, i log e le esportazioni delle regole rendono l'ambiente riproducibile, a patto di rispettare i prerequisiti e le procedure di avvio.

6.3 Limiti

Scenari e campione. Gli scenari sono sintetici e svolti in un ambiente controllato, con una topologia ridotta, servizi semplici e traffico generato apposta. Il confronto V1/V2 si basa su quattro casi: documenta il comportamento della logica nei casi considerati, ma non consente di prevederne le prestazioni su traffico eterogeneo.

Soglie. Finestre e punteggi sono parametri di progetto, non valori ottenuti da un addestramento su traffico reale. Una soglia troppo restrittiva può lasciar passare comportamenti sospetti, una troppo permissiva può segnalare attività legittime. Inoltre, una sequenza anomala di richieste non implica necessariamente la presenza di ransomware: il motore riconosce condizioni ritenute sospette, non una compromissione accertata.

Risposta automatica. Il reindirizzamento è stato verificato in una topologia semplice. In una rete reale andrebbero considerati il numero di connessioni, la presenza di più servizi, la gestione delle connessioni già stabilite e l'impatto dei falsi positivi. Anche la durata del timeout andrebbe valutata sul contesto: troppo breve può non contenere un comportamento persistente, troppo lunga può penalizzare una sorgente legittima classificata male.

FileMonitor e correlazione. Le informazioni raccolte non permettono di associare un'operazione sui file a una sorgente di rete. Il correlatore trova eventi vicini nel tempo e confronta gli identificativi disponibili, ma non può dimostrare che una richiesta abbia causato una modifica. Per questo servirebbero strumenti aggiuntivi per osservare processi, connessioni e operazioni eseguite.

Distanza da un attacco reale. Un attacco ransomware reale può comprendere fasi non riprodotte nel laboratorio, tra cui l'esecuzione di codice malevolo, l'accesso a risorse condivise e la cifratura effettiva dei dati. I risultati non costituiscono pertanto una validazione dell'efficacia contro ransomware reali.

6.4 Possibili sviluppi futuri

Tra i possibili sviluppi futuri rientrano: ampliare gli scenari con traffico legittimo più vario e comportamenti sospetti diversi; valutare soglie e punteggi su più dati; misurare il tempo che passa tra la condizione sospetta e l'applicazione della contromisura; aggiungere al FileMonitor informazioni sui processi responsabili e collegarle all'attività di rete; e, infine, centralizzare log e visualizzazione per consultare più facilmente le evidenze.

6.5 Sintesi

La prova integrata ha permesso di osservare l'intera sequenza: il motore riconosce una sequenza di richieste, aggiorna gli insiemi nftables, il traffico viene reindirizzato verso l'honeypot e, allo scadere del timeout, il servizio legittimo viene ripristinato. Il risultato riguarda esclusivamente le condizioni sperimentali considerate. Per estendere la valutazione servirebbero scenari più vari, più prove e strumenti che colleghino gli eventi di rete ai processi che operano sui file.

Capitolo 7

Riproduzione degli esperimenti

7.1 Obiettivo e requisiti

Il presente capitolo descrive la procedura per riprodurre i principali esperimenti, dalla configurazione iniziale fino all'osservazione di rilevamento e risposta automatica. Codice e configurazioni sono nel repository del progetto:

https://github.com/NS-Projects-Unina/Ransomware_IDS_DSP

Le attività si svolgono esclusivamente nelle reti Docker del laboratorio, senza esecuzione di malware reale né cifratura dei documenti. Il laboratorio è stato sviluppato e verificato su Kali Linux; servono Docker Engine, Docker Compose, Git, Python 3, l'utility `nsenter` e privilegi amministrativi per la configurazione delle reti. Docker deve essere in esecuzione e l'utente deve poter gestire i container. Per controllare gli strumenti:

```
docker --version
docker-compose --version
git --version
python3 --version
```

Nelle procedure viene utilizzato il comando `docker-compose`, coerentemente con gli script del progetto.

7.2 Preparazione e avvio

Si clona il repository e si entra nella directory principale, dove si trovano `docker-compose.yml`, gli script, le configurazioni e i programmi Python:

```
git clone https://github.com/NS-Projects-Unina/Ransomware_IDS_DSP.git

cd Ransomware_IDS_DSP
```

Il volume dei documenti è dichiarato come esterno in Docker Compose e va quindi creato prima dell'avvio, per evitare errori durante l'avvio dei servizi. È inoltre necessario predisporre le directory dei log:

```
docker volume create ransomware_demo_data
```

```
mkdir -p logs/suricata
mkdir -p logs/honeypot
```

Successivamente si costruiscono le immagini e si avvia il laboratorio con lo script del progetto, che lancia i container, prepara rotte e regole nftables e avvia Suricata e il Behavioral Engine sul Gateway:

```
docker-compose build
./scripts/start-lab.sh
```

7.3 Verifiche iniziali

Per controllare che i quattro container (Attacker, Gateaway, Target, FileMonitor) siano attivi e per leggere i log del Gateway:

```
docker-compose ps
docker-compose logs --tail=50 Gateaway
```

Le regole nftables sono organizzate in tre tabelle: `inet lab_filter` per il traffico inoltrato, `inet ips_response` con l'insieme `blocked_v4` e `ip lab_redirect` con l'insieme `suspicious_v4`. Si consultano dal Gateway:

```
docker-compose exec Gateaway nft list ruleset
docker-compose exec Gateaway nft list set ip lab_redirect suspicious_v4
```

Prima della generazione del traffico di test, si verifica che l'insieme non contenga l'indirizzo IP dell'Attacker. Il Target ha indirizzo `10.20.20.2`; dal container Attacker si verifica il servizio legittimo:

```
curl -i http://10.20.20.2:8081/
```

La risposta deve contenere `LAB_LEGITIMATE_SERVICE`.

7.4 Traffico di test e rilevamento

Dal container Attacker, anziché dall'host, viene generata la sequenza di richieste ripetute a documenti diversi:

```
for i in $(seq 1 15); do
    curl -s \
      "http://10.20.20.2:8081/document$i" \
      > /dev/null
done
```

L'attivazione del rilevamento dipende dalle condizioni configurate nel motore e deve pertanto essere verificata nei log del Gateway:

```
docker-compose logs --tail=100 Gateaway
```

Nella prova integrata documentata è stato registrato un evento `fast_10s` con punteggio 100, relativo a 10 richieste GET verso documenti distinti in 10 secondi dalla sorgente `10.10.10.2`. In una nuova esecuzione il risultato va verificato sui log di quella prova.

7.5 Reindirizzamento e timeout

Durante la validità della contromisura, `suspicious_v4` deve contenere `10.10.10.2`:

```
docker-compose exec Gateaway nft list set ip lab_redirect suspicious_v4
```

La regola DNAT manda le nuove connessioni alla porta 8081 sulla 8080 dell'honeypot. Dall'Attacker:

```
curl -i http://10.20.20.2:8081/
```

Con il reindirizzamento attivo la risposta contiene `DOCUMENT-SERVICE/1.0`, indicando che la richiesta è stata gestita dall'honeypot anziché dal servizio legittimo. Il timeout dell'insieme è di un minuto: trascorso tale intervallo, si verifica nuovamente lo stato dell'insieme.

```
sleep 65
docker-compose exec Gateaway nft list set ip lab_redirect suspicious_v4
curl -i http://10.20.20.2:8081/
```

Se non sono arrivati nuovi eventi che mantengano attiva la contromisura, l'indirizzo non è più nell'insieme e la risposta torna a essere `LAB_LEGITIMATE_SERVICE`.

7.6 Evidenze

Le evidenze raccolte nelle verifiche sono nella directory:

`evidence/gateway-autostart/`

Le evidenze comprendono configurazioni, stato dei container, regole nftables ed eventi del motore comportamentale, utilizzabili per confrontare una nuova esecuzione con quella documentata:

- `containers.txt`;
- `gateway.log`;
- `nft-ruleset.txt`;
- `lab-filter.txt`;
- `ips-response.txt`;
- `lab-redirect.txt`;
- `network-behavioral-live-alerts.jsonl`;
- `behavioral-live-response.jsonl`.